LangChain Agents

Section 05: Building Autonomous Reasoning Systems

LangChain v1.2.x (March 2026)

Mental Model: Agents as Reasoning Loops

An **agent** is an LLM that:

1. Thinks about what to do next
2. Decides which tool(s) to call
3. Observes the tool results
4. Repeats until it has an answer

Unlike chains (linear), agents are **dynamic**:

- Chain: "Always call API, then summarize"
- Agent: "Analyze the query, then decide what tools I need"

Key Insight

Agents are control flow determined by the LLM, not by you.

What is an Agent? (Formal Definition)

Definition: An agent is a system that:

1. Takes a user query or goal as input
2. Uses an LLM to reason about which **tools** to invoke
3. Calls one or more tools with appropriate arguments
4. Observes the tool outputs
5. Iterates until it reaches a terminal state
6. Returns a final answer to the user

In LangChain v1.x:

- Agents are built on **LangGraph** under the hood
- `create_react_agent()` is the main constructor
- Returns a `CompiledGraph`—a runnable state machine
- State tracks messages, tool calls, and observations

[Docs: langchain.agents]

The ReAct Pattern: Reason → Act → Observe

ReAct = **Re**asoning + **Act**ing

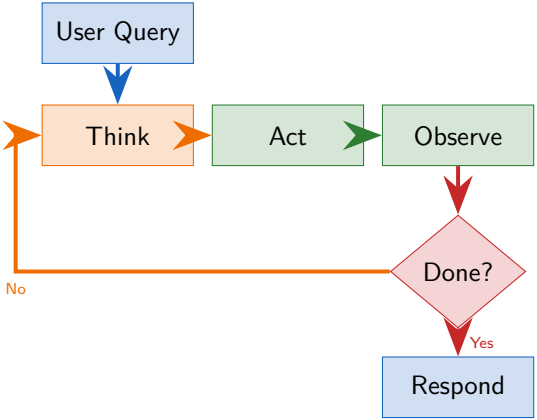
Step	What Happens
Reason	LLM thinks: "I need to do X"
Act	Agent calls tool(s) based on that reasoning
Observe	Tool results are captured and fed back
Think	LLM sees results, decides next step
	Repeat until done

Advantages:

- Transparent reasoning (you see the LLM's thoughts)
- Robust error recovery (LLM can react to failures)
- Flexible—handles new tools and tasks without retraining

[Docs: Yao et al., ReAct: Synergizing Reasoning and Acting in Language Models]

Agent Loop Diagram



The agent iterates through Think → Act → Observe until it decides it's done.

Static vs Dynamic Agents in LangChain v1.2.x

Static Agents

- Tools are fixed at setup
- Runs the same way every time
- Predictable, testable
- Example: calculator + web search

Use when:

- You have a known set of tools
- Task is well-defined

Dynamic Agents

- Tools can change at runtime
- More flexible, adaptive
- Harder to debug
- Example: plugin-based system

Use when:

- Tools are user-provided
- Extensible system needed

LangChain v1.2.x

`create_react_agent()` supports both—pass tools at initialization.

create_react_agent() — The Main Constructor

```python
from langchain.agents import create_react_agent
from langchain_openai import ChatOpenAI
from langchain_core.tools import tool

llm = ChatOpenAI(model="gpt-4", temperature=0)

# Define tools
@tool
def calculator(expression: str) -> str:
    """Evaluate a math expression."""
    return str(eval(expression))

# Create the agent
tools = [calculator]
agent = create_react_agent(
    llm=llm,
    tools=tools,
    prompt=None  # Uses default ReAct prompt
)
```

What you get: A `CompiledGraph` (LangGraph runnable)

[Docs: langchain.agents.create_react_agent]

Running the Agent

```python
from langchain.agents import AgentExecutor

# Wrap in AgentExecutor for convenience
executor = AgentExecutor(agent=agent, tools=tools)

# Invoke it
result = executor.invoke({
    "input": "What is 42 + 8?"
})

print(result["output"])
# Output: The answer is 50.
```

Key points:

- `AgentExecutor` wraps the agent graph
- `invoke()` blocks until done
- State dict contains `input` (query) and `output` (result)
- Messages are tracked internally

[Docs: langchain.agents.AgentExecutor]

Complete Working Example: ReAct Agent

```python
from langchain.agents import create_react_agent, AgentExecutor
from langchain_openai import ChatOpenAI
from langchain_core.tools import tool
import math

llm = ChatOpenAI(model="gpt-4", temperature=0)

@tool
def add(a: float, b: float) -> float:
    """Add two numbers."""
    return a + b

@tool
def multiply(a: float, b: float) -> float:
    """Multiply two numbers."""
    return a * b

tools = [add, multiply]
agent = create_react_agent(llm, tools)
executor = AgentExecutor(agent=agent, tools=tools)

# Use it
result = executor.invoke({"input": "Calculate 10 + 5 * 2"})
print(result["output"])
```

Agent State and Message History

The agent maintains a **message history**:

- **User message:** Your question
- **AI message (Thought):** LLM's reasoning
- **Tool call:** Which tool to invoke + args
- **Tool message:** Result from the tool
- (repeat Thought → Tool call → Tool message)
- **Final AI message:** The answer

Access state:

```
input":  "..."
Via intermediate steps intermediate = result.get("intermediate_steps", [])for step in intermediate :
    tool_name, tool_input, tool_output = step print(tool_name + " : " + tool_output)
```

Streaming Agent Output with astream_events()

Stream events as the agent thinks and acts:

```
async def stream_agent_output():
    async for event in executor.astream_events(
        {"input": "What is 5 + 3?"},
        version="v1"
    ):
        kind = event["event"]
        if kind == "on_llm_start":
            print("LLM thinking...")
        elif kind == "on_tool_start":
            print(f"Calling {event['data']...}")
        elif kind == "on_tool_end":
            print(f"Tool returned: {event['data']...}")
        elif kind == "on_chain_end":
            print("Agent done!")

# Run it
import asyncio
asyncio.run(stream_agent_output())
```

Use for: Real-time UI updates, monitoring long-running agents

Human-in-the-Loop: Interrupts and Confirmations

Pause agent execution to ask for approval:

```python
# Configure to interrupt before tool calls
agent_with_tools = create_react_agent(llm, tools)

def approval_fn(state):
    """Ask user before executing a tool."""
    tool_name = state["messages"][-1].tool_calls[0]["name"]
    approved = input(f"Approve {tool_name}? [y/n]: ")
    return approved.lower() == "y"

# Manually step through execution
state = {"input": "Delete all files..."}
for event in agent_with_tools.stream(state):
    if should_interrupt(event):
        if not approval_fn(event):
            print("Tool rejected by user")
            break
```

Use for: Sensitive operations, safety-critical systems, user control

Agent with Structured Output

Force agent to return structured output using tool schemas:

```
from pydantic import BaseModel
from typing import List

class PlanOutput(BaseModel):
    steps: List[str]
    estimated_time_mins: int

# Tool with structured output
@tool
def create_plan(goal: str) -> PlanOutput:
    """Create a structured plan."""
    return PlanOutput(
        steps=["Step 1", "Step 2"],
        estimated_time_mins=30
    )

tools = [create_plan]
agent = create_react_agent(llm, tools)
result = executor.invoke({"input": "Plan a trip"})

# Structured output returned
```

Multi-Tool Agents with Complex Reasoning

```python
@tool
def search_web(query: str) -> str:
    """Search the web."""
    return f"Results for '{query}'"

@tool
def get_weather(location: str) -> str:
    """Get current weather."""
    return f"Weather in {location}: Sunny"

@tool
def send_email(to: str, subject: str, body: str) -> str:
    """Send an email."""
    return f"Email sent to {to}"

tools = [search_web, get_weather, send_email]
agent = create_react_agent(llm, tools)
executor = AgentExecutor(agent=agent, tools=tools)

# Agent picks the right tools in order
result = executor.invoke({
    "input": "Check weather in NYC and email results"
})
print(result["output"])
```

Controlling Agent Behavior: Prompts, Iterations, Stopping

Custom system prompt:

```
from langchain_core.prompts import ChatPromptTemplate

system_prompt = "You are a helpful assistant..."
prompt = ChatPromptTemplate.from_messages([
    ("system", system_prompt),
    ("user", "{input}"),
])

agent = create_react_agent(llm, tools, prompt=prompt)
```

Limit iterations:

```
executor = AgentExecutor(
    agent=agent,
    tools=tools,
    max_iterations=5,  # Stop after 5 tool calls
    early_stopping_method="force"  # Force stop
)
```

[Docs: create_react_agent, AgentExecutor parameters]

Error Handling in Agents

Handle tool failures gracefully:

```
from langchain.tools import Tool

def safe_search(query: str):
    try:
        return search(query)
    except Exception as e:
        return f"Search failed: {e}"

search_tool = Tool(
    name="search",
    func=safe_search,
    description="Search the web safely"
)

executor = AgentExecutor(
    agent=agent,
    tools=[search_tool],
    handle_parsing_errors=True  # Recover from parse errors
)
```

Common Issues

Agent Debugging and Tracing with LangSmith

Enable tracing to debug agent behavior:

```
import os
os.environ["LANGCHAIN_TRACING_V2"] = "true"
os.environ["LANGCHAIN_API_KEY"] = "your_key"
os.environ["LANGCHAIN_PROJECT"] = "my-agent-project"

# Now all agent runs are traced
result = executor.invoke({"input": "What is 2 + 2?"})

# View traces at https://smith.langchain.com
```

LangSmith shows:

- Full execution trace (every LLM call, tool call)
- Latency breakdown
- Token usage
- Error traces
- Comparison of runs

[Docs: LangSmith tracing]

When to Use Agents vs Chains

Aspect	Chain	Agent
Flow	Fixed, linear	Dynamic, reasoning-based
Tool use	Hardcoded sequence	LLM decides
Flexibility	Low	High
Latency	Predictable	Variable
Complexity	Low	Higher
Debugging	Easier	Harder

Use Chains when:

- Flow is well-defined (e.g., "fetch data → transform → summarize")
- You need predictable latency
- Low error rates are critical

Use Agents when:

- Problem requires reasoning ("what tool do I need?")
- Tool sequence is unknown
- Adaptability matters

Common Pitfalls and How to Avoid Them

Pitfall 1: Infinite Loops

Agent keeps calling the same tool over and over.
Fix: Set `max_iterations`, use early stopping, improve tool descriptions.

Pitfall 2: Poor Tool Descriptions

Agent picks wrong tool because description is vague.
Fix: Write clear, precise tool docs with examples.

Pitfall 3: Missing Error Handling

Tool fails, agent halts or gives bad answer.
Fix: Wrap tools with try/catch, return informative error messages.

Pitfall 4: Token Overflow

Agent uses too many tokens and context window fills up.

Production-Ready Agent Pattern

A robust pattern for production systems:

```
from langchain.agents import create_react_agent, AgentExecutor
from tenacity import retry, stop_after_attempt

class ProductionAgent:
    def __init__(self, llm, tools):
        self.agent = create_react_agent(llm, tools)
        self.executor = AgentExecutor(
            agent=self.agent,
            tools=tools,
            max_iterations=10,
            early_stopping_method="force",
            handle_parsing_errors=True
        )

    @retry(stop=stop_after_attempt(3))
    def run(self, query: str, timeout: int = 30) -> str:
        try:
            result = self.executor.invoke(
                {"input": query},
                config={"timeout": timeout}
            )
            return result["output"]
```

Async Agents for Concurrency

Use async for non-blocking operations:

```
async def async_agent_example():
    # Use async tools
    @tool
    async def fetch_data(url: str) -> str:
        async with aiohttp.ClientSession() as s:
            async with s.get(url) as r:
                return await r.text()

    tools = [fetch_data]
    agent = create_react_agent(llm, tools)
    executor = AgentExecutor(agent=agent, tools=tools)

    # Run asynchronously
    result = await executor.ainvoke({"input": "Fetch..."})
    return result["output"]

# Call it
import asyncio
output = asyncio.run(async_agent_example())
```

[Docs: ainvoke, async tools]

Agent Middleware and Hooks

Intercept and modify agent behavior:

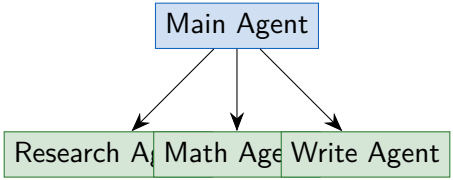
```
# Log every tool call
from langchain.callbacks import BaseCallbackHandler

class LoggingCallback(BaseCallbackHandler):
    def on_tool_start(self, **kwargs):
        print(f"Tool started: {kwargs['tool']}")

    def on_tool_end(self, output, **kwargs):
        print(f"Tool output: {output[:50]}...")

executor = AgentExecutor(agent=agent, tools=tools)
result = executor.invoke(
    {"input": "What is 5 + 3?"},
    config={"callbacks": [LoggingCallback()]}
)
```

Hooks: `on_llm_start`, `on_tool_start`, `on_chain_end`, etc.

[Docs: CallbackHandler]

Hierarchical Agents: Agent Teams

Idea: Use agents as tools for other agents.



Benefits:

- Modular: each agent has one responsibility
- Scalable: add agents without retraining
- Debuggable: trace each sub-agent

```
input":  query
tools = [run_research_agent]main_agent = create_react_agent(llm, tools)
```

Memory in Agents

Agents have implicit memory via message history:

```
# Each invoke call includes all prior messages
executor = AgentExecutor(agent=agent, tools=tools)

# First query
result1 = executor.invoke({"input": "My name is Alice"})

# Second query (with memory)
result2 = executor.invoke({"input": "What is my name?"})
# Agent remembers: Alice
```

Explicit memory with chat history:

```
from langchain_community.chat_message_histories import SQLChatMessageHistory

history = SQLChatMessageHistory(
    session_id="user-123",
    connection_string="sqlite:///memory.db"
)

# Auto-load and save history
agent.with_message_history(history).invoke(...)
```

[Docs: Memory integration]

Tool Validation and Schemas

Define strict schemas for tools:

```
from pydantic import BaseModel, Field

class SearchInput(BaseModel):
    query: str = Field(description="Search query")
    max_results: int = Field(10, description="Max results")

@tool
def search(input: SearchInput) -> str:
    """Search the web with validation."""
    return f"Results for {input.query}"

# LLM respects the schema
tools = [search]
agent = create_react_agent(llm, tools)

# Agent passes correctly typed arguments
executor.invoke({"input": "Search for cats with 5 results"})
```

Benefits:

- Type-safe tool calls
- LLM gets clear argument specs

Tool Error Recovery

Handle tool failures and let agent retry:

```
from langchain_core.tools import tool

@tool
def unstable_api(query: str) -> str:
    """May fail sometimes."""
    import random
    if random.random() < 0.5:
        raise Exception("API temporarily down")
    return f"Result: {query}"

# Agent automatically retries on tool error
tools = [unstable_api]
executor = AgentExecutor(
    agent=agent,
    tools=tools,
    handle_parsing_errors=True
)

result = executor.invoke({"input": "Call API"})
# Agent may retry or use alternative approach
```

Key: Tools should return error messages, not throw exceptions.

Agent Observability and Monitoring

Key metrics to track in production:

- **Success rate:** % of queries answered correctly
- **Tool accuracy:** How often tools give right results
- **Latency:** Time per query (and per iteration)
- **Token usage:** Tokens consumed (cost)
- **Max iterations hit:** % of runs hitting iteration limit
- **Tool call distribution:** Which tools used most
- **Error rate:** % of runs with errors

Tools:

- **LangSmith:** Full tracing and analytics
- **Custom logging:** Write metrics to database
- **APM (Datadog, New Relic):** Integration hooks

Advanced: Conditional Logic via Tool Descriptions

Guide agent behavior through tool descriptions:

```
@tool
def search_internal(query: str) -> str:
    """Search only internal documents. Use for proprietary info."""
    return f"Internal result: {query}"

@tool
def search_web(query: str) -> str:
    """Search public web. Use for general knowledge."""
    return f"Web result: {query}"

# LLM reads descriptions and picks appropriate tool
tools = [search_internal, search_web]
agent = create_react_agent(llm, tools)

# For internal query, LLM chooses search_internal
# For general query, LLM chooses search_web
executor.invoke({"input": "Our Q4 revenue..."})
```

Key: Clear descriptions = better tool selection.

Tool Composition: Tools that Call Tools

Tools can be built from other tools:

```
@tool
def compound_search(query: str) -> str:
    """Search web and email simultaneously."""
    web = search_web(query)
    email = search_email(query)
    return f"Web: {web}\nEmail: {email}"

# This tool is itself a tool that agent can call
tools = [compound_search, search_web, search_email]
agent = create_react_agent(llm, tools)
```

Benefits:

- Reduce agent decision-making
- Pre-compose complex workflows
- Hide implementation details

Agents: Quick Cheat Sheet

Create an agent:

```
from langchain.agents import create_react_agent, AgentExecutor
agent = create_react_agent(llm, tools)
executor = AgentExecutor(agent=agent, tools=tools)
```

Run it:

```
result = executor.invoke({"input": "Your query"})
output = result["output"]
```

Stream it:

```
async for event in executor.astream_events({"input": "..."}, version="v1"):
    print(event)
```

Key configs:

```
AgentExecutor(agent=agent, tools=tools,
              max_iterations=5, early_stopping_method="force",
              handle_parsing_errors=True)
```

Summary: Section 05 - Agents

Core Concepts:

1. Agents are LLMs that reason and decide which tools to use
2. ReAct pattern: Reason $\rightarrow$ Act $\rightarrow$ Observe $\rightarrow$ Repeat
3. `create_react_agent()` builds agents on LangGraph

Implementation:

1. Define tools with clear descriptions
2. Create agent with `create_react_agent(llm, tools)`
3. Run via `AgentExecutor.invoke()` or stream

Production Readiness:

1. Set iteration limits, error handling, timeouts
2. Use LangSmith for debugging and observability
3. Test agents thoroughly (infinite loops, edge cases)

When to use agents: Problems requiring reasoning and dynamic tool selection.

Self-Check: Test Your Understanding

Question 1: What is the core difference between a chain and an agent?

Question 2: Describe the ReAct pattern in one sentence.

Question 3: What does `create_react_agent()` return?

Question 4: Why is a good tool description important?

Question 5: Name 3 common pitfalls when building agents.

Question 6: How do you enable tracing for debugging?

Question 7: What is the purpose of `max_iterations` in AgentExecutor?

Question 8: When would you choose an agent over a chain?

Suggested Answers

Q1: Chains have fixed, linear flow. Agents use LLM to decide tool sequence dynamically.

Q2: Agent thinks about a step, acts (calls a tool), observes the result, then repeats until done.

Q3: A `CompiledGraph` (LangGraph runnable state machine).

Q4: Poor descriptions cause agents to pick wrong tools or fail to recognize which tool to use.

Q5: (1) Infinite loops, (2) Poor tool descriptions, (3) Missing error handling, (4) Token overflow, (5) Hallucinated tools.

Q6: Set `LANGCHAIN_TRACING_V2=true` and `LANGCHAIN_API_KEY` environment variables.

Q7: Prevent infinite loops by halting execution after N iterations.

Q8: When the task requires reasoning, tool selection is unknown, or multiple tool paths are viable.

Further Learning Resources

Official Documentation:

- `langchain.agents` module
- LangGraph quickstart (agents built on LangGraph)
- Tool integration examples

Recommended Patterns:

- Hierarchical agents (agent teams)
- Human-in-the-loop with confirmations
- Structured output agents

Debugging Tools:

- LangSmith for tracing
- `astream_events()` for streaming
- Callback handlers for custom logging

[Docs: langchain.agents, LangGraph, LangSmith]

Next Steps

After this section, you can:

1. Build a functional ReAct agent with custom tools
2. Debug agents using LangSmith
3. Handle errors and edge cases
4. Deploy agents safely to production

Practice exercise:

1. Create a weather agent with tools for:
   - Get current weather
   - Get forecast
   - Convert units
2. Test with: "What will the weather be in NYC this weekend in Celsius?"
3. Trace the agent execution with LangSmith

Section 06 preview: Multi-agent systems and LangGraph state machines.